



Sistemas Embebidos I

Otoño 2026

Práctica 4: Interrups

Camila Rodríguez Rosas

Ana Karen Rojas Ledesma

Jonathan Hernández Lazcano

05/09/2026

Índice

Práctica 4:	2
Introducción	2
Descripción de la práctica.....	2
Objetivo.....	2
Marco teórico.....	2
Desarrollo	3
Materiales	3
Esquemático	4
Desarrollo	5
Resultados	7
Evidencias	7
Conclusiones	8
Declaración de IA	8
Referencias	8

Práctica 4:

Introducción

Descripción de la práctica

La siguiente práctica consiste en implementar y analizar una **interrupción de GPIO (IRQ)** en un sistema embebido. El propósito es observar de manera experimental el comportamiento de una interrupción desde que ocurre el evento externo hasta que comienza y termina la rutina de servicio de interrupción (ISR).

Se utilizan dos señales en el osciloscopio. El Canal 1 representa el evento que provoca la interrupción, mientras que el Canal 2 corresponde a una señal de traza generada por el programa al entrar y salir de la ISR. De esta manera es posible observar y medir la diferencia temporal entre el evento y el inicio de la ISR, así como la duración de la ejecución de la propia ISR.

Objetivo

Implementar una interrupción de GPIO y analizar experimentalmente su comportamiento mediante el uso de una señal de traza, con el propósito de medir la latencia de interrupción y el tiempo de servicio de la ISR utilizando un osciloscopio.

Marco teórico

Interrupciones (IRQ) e ISR

Una interrupción IRQ, Interrupt Request, es un evento que pausa el flujo normal de ejecución del programa para atender una función específica conocida como ISR o Interrupt Service Routine por sus siglas en inglés. Una vez terminado el ISR, el procesador regresa al punto donde se encontraba ejecutando el código de fondo.

Para que una interrupción se pueda ejecutar, es necesario que estén habilitados el permiso de la fuente específica que genera el evento y la línea correspondiente dentro del NVIC. Si alguno está deshabilitado, la interrupción no llega hasta la CPU.

Flujo de atención de una interrupción

Cuando ocurre una interrupción, primero se genera el evento y la IRQ queda pendiente, posteriormente la CPU interrumpe temporalmente el código que se estaba ejecutando y automáticamente guarda determinados registros. Luego consulta la *Vector Table* donde se encuentra asociada la dirección de la función que debe ejecutarse.

Después de localizar la ISR, esta se ejecuta y al terminar, el procesador recupera el contexto que había guardado y continúa con el programa de fondo desde el punto de su interrupción.

Latencia de interrupción

Es el tiempo que transcurre desde que la IRQ es válida hasta que comienza la ejecución de la primera instrucción de la ISR, esta incluye el proceso de entrada a la interrupción. Esto se observa en la práctica mediante dos señales, la que muestra el evento físico, y la segunda que muestra el momento en el que inicia la ISR, la distancia temporal entre ambos corresponde a la latencia.

Tiempo de servicio de la ISR

Es el tiempo durante el cual se ejecuta la ISR, desde la primera hasta la última instrucción, en este caso, el ancho de pulso observado en el osciloscopio representa de manera directa el tiempo de servicio de la ISR.

IRQ de GPIO mediante polaridad

Una interrupción de GPIO puede configurarse para responder ante diferentes tipos de eventos, que son: GPIO_IRQ_EDGE_FALL: flanco de bajada, GPIO_IRQ_EDGE_RISE: flanco de subida, GPIO_IRQ_LEVEL_HIGH: nivel alto, GPIO_IRQ_LEVEL_LOW: nivel bajo. Y se revisan las siguientes formas de disparo: edge-triggered y level-triggered.

En la forma edge-triggered, la interrupción ocurre cuando existe una transición, por ejemplo, de bajo a alto o de alto a bajo, es apropiado para eventos discretos como un botón o un flanco de reloj. Por otra parte, en level-triggered la interrupción se mantiene asociada mientras una condición permanezca activa.

Prioridad de interrupciones

En arquitecturas Cortex-M, un número de prioridad más bajo significa mayor urgencia, se manejan 16 niveles lógicos de prioridad, permitiendo que diferentes interrupciones compitan por la atención del procesador.

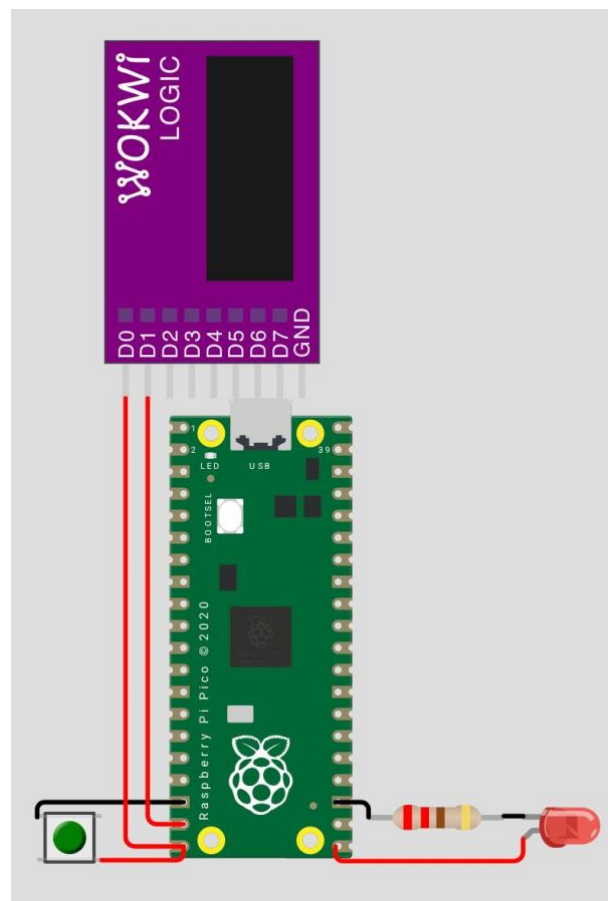
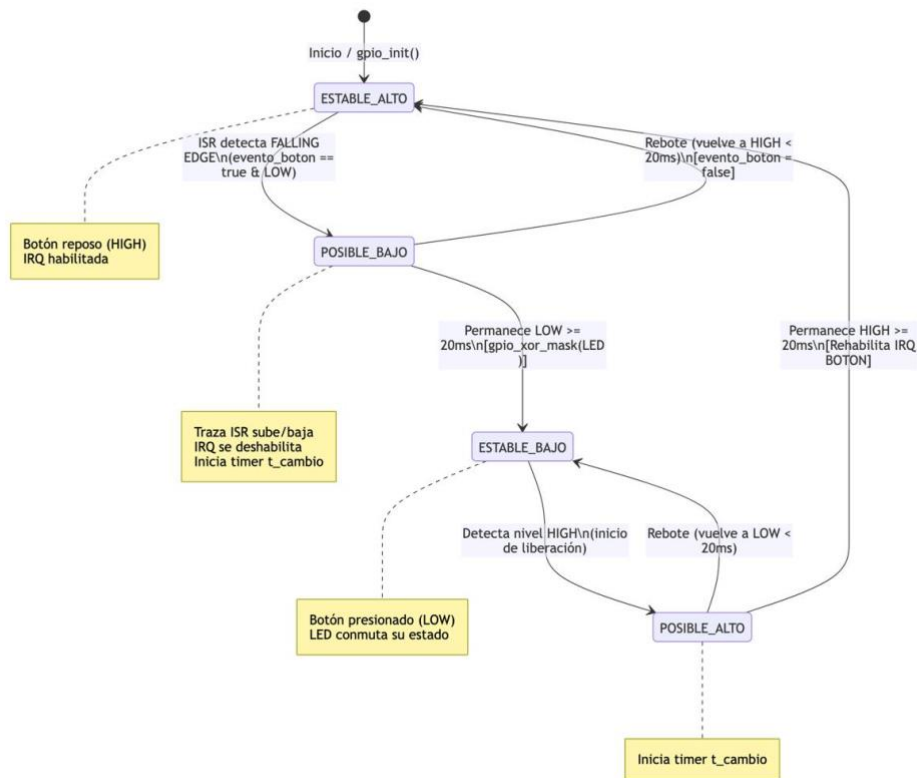
Desarrollo

Materiales

1. Raspberry Pi Pico 2 W
2. Protoboard
3. LED rojo
4. Push button
5. Jumpers

Software

1. Visual Studio Code
2. Wokwi
3. Surfer project



Desarrollo

```

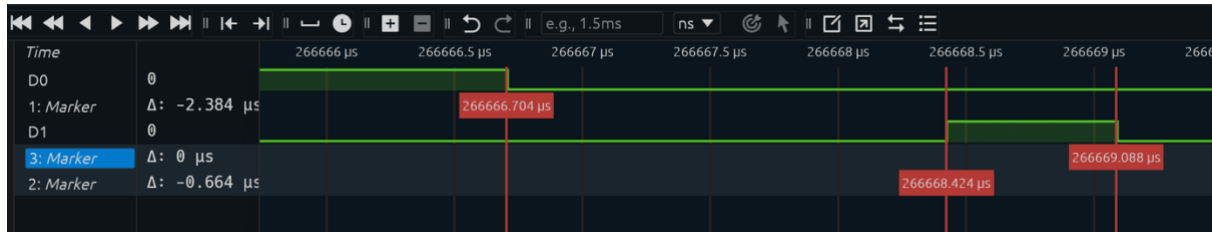
1  #include "pico/stdlib.h"
2  #include "hardware/gpio.h"
3
4  #define TRAZA 14
5  #define BOTON 15
6  #define LED 16
7
8  #define VENTANA_MS 20
9
10 volatile bool evento_boton = false;
11
12
13 static void button_isr(uint gpio, uint32_t events){
14 if (gpio == BOTON && (events & GPIO_IRQ_EDGE_FALL)){
15 //Inicio de la ISR: Traza en alto
16 gpio_put(TRAZA, 1);
17
18 /* IMPORTANTE: Deshabilitamos inmediatamente nuevas IRQ provenientes del botón.
19 * Así, los rebotes del botón NO pueden volver a ejecutar la ISR.*/
20 gpio_set_irq_enabled(BOTON, GPIO_IRQ_EDGE_FALL, false);
21
22 gpio_acknowledge_irq(BOTON, GPIO_IRQ_EDGE_FALL);
23 evento_boton = true;
24
25 //Fin de la ISR: Traza en bajo
26 gpio_put(TRAZA, 0);
27 }
28 }
29
30 //Estados del Debounce
31 typedef enum
32 {
33 ESTABLE_ALTO,
34 POSIBLE_BAJO,
35 ESTABLE_BAJO,
36 POSIBLE_ALTO
37 } estado_t;
38
39
40 int main(void)
41 {
42 stdio_init_all();
43
44 gpio_init(TRAZA);
45 gpio_set_dir(TRAZA, GPIO_OUT);
46 // Estado normal fuera de una IRQ
47 gpio_put(TRAZA, 0);
48
49 gpio_init(LED);
50 gpio_set_dir(LED, GPIO_OUT);
51 gpio_put(LED, 0);
52
53 gpio_init(BOTON);
54 gpio_set_dir(BOTON, GPIO_IN);
55 gpio_pull_up(BOTON);
56
57 //Configuración de la IRQ
58 gpio_set_irq_enabled_with_callback(BOTON, GPIO_IRQ_EDGE_FALL, true, &button_isr);
59
60 //Estado del Debounce
61 estado_t estado = ESTABLE_ALTO;
62 absolute_time_t t_cambio;

```

```

1  while (true){
2
3  bool nivel_alto = gpio_get(BOTON);
4
5
6  switch (estado){
7
8  //Boton sin presionar
9  case ESTABLE_ALTO:
10
11  //Si hay posible pulsación
12  if (evento_boton && !nivel_alto){
13  evento_boton = false;
14  estado = POSIBLE_BAJO;
15  t_cambio = get_absolute_time();
16  }
17  break;
18
19  //Posible pulsación
20  case POSIBLE_BAJO:
21
22  // Si regresó a HIGH antes de 20 ms, fue un rebote o una pulsación inválida.
23  if (nivel_alto){
24  estado = ESTABLE_ALTO;
25  evento_boton = false;
26  gpio_acknowledge_irq(BOTON, GPIO_IRQ_EDGE_FALL);
27  }
28
29  //Si permanece LOW durante 20 ms, la pulsación es válida.
30  else if (absolute_time_diff_us(t_cambio, get_absolute_time()) >= (VENTANA_MS * 1000))
31  estado = ESTABLE_BAJO;
32  //Cambiar estado del LED
33  gpio_xor_mask(1u << LED);
34  }
35  break;
36
37  //Botón pulsado
38  case ESTABLE_BAJO:
39
40  //Posible liberación del botón
41  if (nivel_alto){
42  estado = POSIBLE_ALTO;
43  t_cambio = get_absolute_time();
44  }
45  break;
46
47  //Posible liberación
48  case POSIBLE_ALTO:
49
50  //Si vuelve a LOW antes de 20 ms, todavía existe rebote.
51  if (!nivel_alto){
52  estado = ESTABLE_BAJO;
53  }
54
55  //Si permanece HIGH durante 20 ms, confirmamos la liberación.
56  else if (absolute_time_diff_us(t_cambio, get_absolute_time()) >= (VENTANA_MS * 1000))
57  estado = ESTABLE_ALTO;
58  evento_boton = false;
59  gpio_acknowledge_irq(BOTON, GPIO_IRQ_EDGE_FALL);
60  // Volver a permitir una nueva IRQ.
61  gpio_set_irq_enabled(BOTON, GPIO_IRQ_EDGE_FALL, true);
62  }
63  break;
64  }
65  tight_loop_contents();
66  }
67  }

```



Resultados

- D_0 : Botón
- D_1 : Traza

Latencia Experimental:

$$t_{LATENCIA} = t_{TRAZA} - t_{BOTÓN} = 266668.424\mu s - 266666.704\mu s$$

$$t_{LATENCIA} = 1.72\mu s$$

Tiempo de servicio experimental:

$$t_{SERVICIO} = t_{TRAZA_FINAL} - t_{TRAZA_INICIAL} = 266669.088\mu s - 266668.424\mu s$$

$$t_{SERVICIO} = 0.664\mu s$$

Parámetro	Valor esperado	Valor medido (Surfer Project)
Latencia de Interrupción	Bajo, del orden de microsegundos	$1.72\mu s$
Tiempo de servicio	Depende del trabajo dentro de la ISR	$0.664\mu s$

Evidencias

Simulación en Wokwi: <https://wokwi.com/projects/474299716347463681>

El siguiente enlace tiene una accesibilidad hasta el 5 de Octubre del 2025. Pero también se puede encontrar el video de evidencia dentro del zip adjunto a la entrega de la práctica.

Evidencia: [SE1_P4_EQ3_EVIDENCIA.mov](#)

Conclusiones

Se logró implementar y caracterizar experimentalmente una interrupción de GPIO en la Raspberry Pi Pico, verificando de manera práctica los conceptos teóricos revisados sobre el modelo de interrupciones en arquitecturas Cortex-M. Mediante la configuración de una IRQ *edge-triggered* (GPIO_IRQ_EDGE_FALL) asociada al botón, y el uso de una señal de traza en un pin auxiliar, fue posible medir directamente en el Surfer Project tanto la latencia de interrupción como el tiempo de servicio de la ISR.

Los resultados obtenidos muestran una latencia de interrupción de **1.72 μ s** y un tiempo de servicio de **0.664 μ s**, valores consistentes con el orden de magnitud esperado (microsegundos) para un procesador Cortex-M0+ operando sin contención significativa de bus ni instrucciones largas en ejecución al momento del evento. Esto confirma que, tal como establece el marco teórico, la latencia depende principalmente del proceso de entrada a la interrupción (apilado de registros y consulta a la *Vector Table*), mientras que el tiempo de servicio está directamente relacionado con la cantidad de trabajo realizado dentro de la ISR.

Declaración de IA

1. ChatGPT
 - a. Comentado de códigos
2. Lucid AI
 - a. Realización del diagrama de flujo mediante código Mermaid
3. Gemini
 - a. Generación del código Mermaid para Lucid AI basado en el código en C
4. Claude
 - a. Búsqueda de información para el marco teorico y referencias.

Referencias

Jonathan: Raspberry Pi Ltd. (2025). RP2040 datasheet: A microcontroller by Raspberry Pi (build-date 2025-02-20; formerly Raspberry Pi (Trading) Ltd.).

<https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>

Jonathan: Adams, V. H. (s. f.). Direct digital synthesis (DDS) in a timer interrupt on the RP2040 [Material del curso ECE 4760]. Cornell University.

https://vanhunteradams.com/Pico/TimerIRQ/SPI_DDS.html

Jonathan: ARM Software. (s. f.). CMSIS-Core (Cortex-M): Interrupts and exceptions (NVIC). ARM.

https://armsoftware.github.io/CMSIS_6/latest/Core/group__NVIC__gr.html

Jonathan: Yiu, J. (2016, 1 de abril). A beginner's guide on interrupt latency — and interrupt latency of the ARM Cortex-M processors [Entrada de blog]. ARM Community.

<https://developer.arm.com/community/arm-community-blogs/b/architectures-and-processors-blog/posts/beginner-guide-on-interrupt-latency-and-interrupt-latency-of-the-arm-cortex-m-processors>

Jonathan: Raspberry Pi Ltd. (s. f.). Raspberry Pi Pico C/C++ SDK — hardware_irq y hardware_gpio [Documentación de la API].

<https://www.raspberrypi.com/documentation/pico-sdk/>

Jonathan: ARM Limited. (s. f.). Cortex-M0 devices generic user guide: The Cortex-M0 processor — Exception model — Vector table. ARM Developer.

<https://developer.arm.com/documentation/dui0497/a/>